Pattern Class (java.util.regex.Pattern)

Java 8+

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`Pattern.compile(regex)`	Compiles the given regular expression string into a reusable Pattern object **EXAMPLE:** `Pattern p = Pattern.compile("\\d+");`
`Pattern.compile(regex, flags)`	Compiles pattern with one or more bitwise-OR flags (e.g. Pattern.CASE_INSENSITIVE) **EXAMPLE:** `Pattern p = Pattern.compile("abc", Pattern.CASE_INSENSITIVE);`
`Pattern.matches(regex, input)`	Quick one-liner check if regex matches the entire input CharSequence **EXAMPLE:** `Pattern.matches("\\d{4}", "2026") → true`
`pattern.matcher(input)`	Creates a Matcher object that will match the given input against this pattern **EXAMPLE:** `Matcher m = p.matcher(inputSequence);`
`pattern.split(input)`	Splits the input around matches of this pattern into a String[] array **EXAMPLE:** `p.split("apple,banana,cherry") → String[3]`
`pattern.split(input, limit)`	Splits input with threshold limit on the number of matching substrings **EXAMPLE:** `p.split("a:b:c:d", 2) → ["a", "b:c:d"]`
`pattern.splitAsStream(input)`	Creates a Stream<String> from the input split around matches (Java 8+) **EXAMPLE:** `p.splitAsStream(text).forEach(System.out::println);`
`pattern.asPredicate()`	Creates a Predicate<String> for filtering streams using matcher.find() (Java 8+) **EXAMPLE:** `list.stream().filter(p.asPredicate()).toList();`
`pattern.asMatchPredicate()`	Creates a Predicate<String> that matches entire string using matcher.matches() (Java 11+) **EXAMPLE:** `list.stream().filter(p.asMatchPredicate()).toList();`
`Pattern.quote(s)`	Returns a literal pattern String wrapped in \Q...\E for exact matching **EXAMPLE:** `Pattern.quote("user[1].id") → "\Quser[1].id\E"`
`pattern.pattern()`	Returns the original regular expression string from which this pattern was compiled **EXAMPLE:** `p.pattern() → "\\d+"`
`pattern.flags()`	Returns this pattern's match flags bitmask **EXAMPLE:** `p.flags() → 2 (Pattern.CASE_INSENSITIVE)`

Matcher Class (java.util.regex.Matcher)

Java 8+

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`m.find()`	Attempts to find the next subsequence of the input sequence that matches the pattern **EXAMPLE:** `while (m.find()) { ... }`
`m.find(start)`	Resets this matcher and attempts to find next match starting from the specified index **EXAMPLE:** `m.find(10) → boolean`
`m.matches()`	Attempts to match the entire input region against the pattern **EXAMPLE:** `m.matches() → true for exact match`
`m.lookingAt()`	Attempts to match the input sequence, starting at the beginning, against the pattern **EXAMPLE:** `m.lookingAt() → true if prefix matches`
`m.group()`	Returns the input subsequence matched by the previous match (group 0) **EXAMPLE:** `m.group() → "2026-08-19"`
`m.group(n)`	Returns the input subsequence captured by the given 1-based group index **EXAMPLE:** `m.group(1) → "2026"`
`m.group(name)`	Returns the input subsequence captured by the named group (?<name>...) (Java 7+) **EXAMPLE:** `m.group("year") → "2026"`
`m.groupCount()`	Returns the number of capturing groups in this matcher's pattern **EXAMPLE:** `m.groupCount() → 2`
`m.start() / m.end()`	Returns start (inclusive) and end (exclusive) character offset of the match **EXAMPLE:** `m.start() → 0, m.end() → 4`
`m.start(group) / m.end(group)`	Returns start / end offset of the subsequence captured by group index or name **EXAMPLE:** `m.start("year") → 0, m.end("year") → 4`
`m.replaceAll(replacement)`	Replaces every subsequence of input sequence that matches pattern with replacement **EXAMPLE:** `m.replaceAll("-") → "hello-world"`
`m.replaceFirst(replacement)`	Replaces first subsequence of input that matches pattern with replacement **EXAMPLE:** `m.replaceFirst("#") → "#123 456"`
`m.replaceAll(function)`	Replaces each match with result of applying function to MatchResult (Java 9+) **EXAMPLE:** `m.replaceAll(mr -> mr.group().toUpperCase())`
`m.results()`	Returns a Stream<MatchResult> of all match results in input order (Java 9+) **EXAMPLE:** `m.results().map(MatchResult::group).toList()`
`m.appendReplacement(sb, repl)`	Non-terminal step: appends preceding text and replacement to StringBuffer/StringBuilder **EXAMPLE:** `m.appendReplacement(sb, newText);`

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`m.appendTail(sb)`	Terminal step: appends remaining input text after last match to StringBuilder (Java 9+) **EXAMPLE:** `m.appendTail(sb);`
`m.reset([input])`	Resets this matcher with optional new input CharSequence **EXAMPLE:** `m.reset("new test string");`
`m.usePattern(newPattern)`	Changes the Pattern that this Matcher uses to find matches **EXAMPLE:** `m.usePattern(Pattern.compile("\\w+"));`

String Class Regex Methods

Java 8+

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`str.matches(regex)`	Returns true if and only if the entire string matches the given regular expression **EXAMPLE:** `"12345".matches("\\d+") → true`
`str.replaceAll(regex, repl)`	Replaces each substring matching regex with replacement string (supports $1, ${name}) **EXAMPLE:** `"a1b2c3".replaceAll("\\d", "#") → "a#b#c#"`
`str.replaceFirst(regex, repl)`	Replaces first substring matching regex with replacement string **EXAMPLE:** `"foo 1 2".replaceFirst("\\d", "#") → "foo # 2"`
`str.split(regex)`	Splits string around matches of regex; trailing empty strings are discarded **EXAMPLE:** `"a,b,,c".split(",") → ["a", "b", "", "c"]`
`str.split(regex, limit)`	Splits string around matches with control on result array length (limit < 0 retains trailing empty strings) **EXAMPLE:** `"a:b:c:".split(":", -1) → ["a", "b", "c", ""]`

Java Pattern Flags

Java 8+

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`Pattern.CASE_INSENSITIVE`	Enables case-insensitive matching (Inline modifier: (?i) - ASCII by default; combine with UNICODE_CASE for full Unicode) **EXAMPLE:** `Pattern.compile("cat", Pattern.CASE_INSENSITIVE)`
`Pattern.MULTILINE`	^ and $ match just after / before line terminators, as well as start / end of input (Inline modifier: (?m)) **EXAMPLE:** `Pattern.compile("^\\w+", Pattern.MULTILINE)`
`Pattern.DOTALL`	The dot (.) character matches any character including line terminators (\n, \r) (Inline modifier: (?s)) **EXAMPLE:** `Pattern.compile("<div>.*</div>", Pattern.DOTALL)`
`Pattern.COMMENTS`	Permits whitespace and comments starting with # inside pattern until end of line (Inline modifier: (?x)) **EXAMPLE:** `Pattern.compile("\\d{3} # area", Pattern.COMMENTS)`
`Pattern.UNICODE_CHARACTER_CLASS`	Enables Unicode version of predefined classes (\w, \d, \s) matching Unicode specs (Java 7+) (Inline modifier: (?U)) **EXAMPLE:** `Pattern.compile("\\w+", Pattern.UNICODE_CHARACTER_CLASS)`
`Pattern.UNICODE_CASE`	Enables Unicode-aware case folding when used alongside Pattern.CASE_INSENSITIVE (Inline modifier: (?u)) **EXAMPLE:** `Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CASE`
`Pattern.UNIX_LINES`	Only the \n character is recognized as a line terminator in ^, $, and . modes (Inline modifier: (?d)) **EXAMPLE:** `Pattern.compile("^line", Pattern.UNIX_LINES)`
`Pattern.CANON_EQ`	Enables canonical equivalence matching (e.g. character with accent vs combining accent) **EXAMPLE:** `Pattern.compile("å", Pattern.CANON_EQ)`
`Pattern.LITERAL`	Specifies that pattern contains literal characters with no metacharacter meanings **EXAMPLE:** `Pattern.compile(".*+?", Pattern.LITERAL)`
`Pattern.CASE_INSENSITIVE | Pattern.MULTILINE`	Combine multiple flags using the bitwise OR (|) operator **EXAMPLE:** `Pattern.CASE_INSENSITIVE | Pattern.MULTILINE`

Java-Specific Syntax, POSIX & Properties

Java 8+

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
`"\\d", "\\b", "\\s"`	Double backslash in Java strings: Java compiler uses 1st escape, regex engine uses 2nd EXAMPLE: `String pat = "\\b[A-Z]+\\b";`

`"""\\d+"""`	Java 15+ Text Blocks (triple quotes) simplify multiline regex, though backslashes still escape
	EXAMPLE: `"""`
	`\\d{4}-\\d{2}-\\d{2}`
	`"""`

`(?<name>X)`	Named capturing group — captures matching text accessible via matcher.group("name") EXAMPLE: `(?<zip>\d{5}) → m.group("zip")`
`\k<name>`	Named backreference within the regex pattern to an earlier named group EXAMPLE: `(?<quote>['"]).*?\k<quote>`
`${name} / $1`	Named and indexed group backreferences in replaceAll() replacement strings EXAMPLE: `m.replaceAll("Found: ${name} (id: $1)")`
`*+`	Possessive quantifier (also ++, ?+, {n,m}+) — eats greedily and never backtracks; prevents ReDoS vulnerabilities EXAMPLE: `"[^"]*+" → matches quoted string fast`
`[a-z&&[def]]`	Character class intersection: matches characters present in both sets ('d', 'e', or 'f') EXAMPLE: `[a-z&&[aeiou]] → vowels only`
`[a-z&&[^bc]]`	Character class subtraction: matches a through z except 'b' and 'c' EXAMPLE: `[0-9&&[^2468]] → odd digits`
`[a-d[m-p]]`	Character class union: matches union of sets (equivalent to [a-dm-p]) EXAMPLE: `[a-c[1-3]] → [a-c1-3]`
`\p{Lower}, \p{Upper}, \p{Digit}`	POSIX character classes: Lowercase [a-z], Uppercase [A-Z], Digits [0-9] EXAMPLE: `\p{Digit}+ → matches numbers`
`\p{Alpha}, \p{Alnum}, \p{Punct}`	POSIX alphabetic [a-zA-Z], alphanumeric, and punctuation characters EXAMPLE: `\p{Alnum}+ → "User1042"`
`\p{Blank}, \p{Space}, \p{Cntrl}`	POSIX space or tab [\t], whitespace [\t\n\x0B\f\r], and control character EXAMPLE: `\p{Blank} → space or tab only`
`\p{javaLowerCase}, \p{javaWhitespace}`	Java character property classes matching Character.isLowerCase() / isWhitespace() EXAMPLE: `\p{javaWhitespace}+`
`\p{IsGreek}, \p{IsLatin}, \p{Sc}`	Unicode Scripts / Blocks and Categories (\p{Sc} for currency symbols: $, €, ¥) EXAMPLE: `\p{Sc}\\d+ → "$100", "€50"`
`\Q...\E`	Literal quotation — all characters between \Q and \E are treated as literal text

PATTERN / SYNTAX	DESCRIPTION & EXAMPLE
	EXAMPLE: `\Q*.[a-z]\E matches literal "*.[a-z]"`
`\R`	Any Unicode line break sequence (\u000D\u000A|\u000A|...) (Java 8+) **EXAMPLE:** `Pattern.compile("\\R") splits any OS newline`
`\A, \Z, \z, \G`	Boundaries: \A (beginning of input), \Z (end except final terminator), \z (strict end of input), \G (end of previous match) **EXAMPLE:** `\\A[a-z]+\\z`